Bild: M. Brown. *Round Silver-colored Chronograph watch*. 2017-07-29. Pexels.com.

CPU-schemaläggning

Jimmy Åhlander

2025-11-16

Mittuniversitetet
MID SWEDEN UNIVERSITY

Agenda

- Schemaläggning:
  - CPU-schemaläggning.
  - CPU-I/O burst cycle.
  - Spärrande och icke-spärrande schemaläggning.
- Algoritmer:
  - Kriterier vid schemaläggning.
  - First-Come, First-Served (FCFS).
  - Shortest Job First (SJF).
  - Prioritetsschemaläggning.
  - Round Robin (RR).
  - Multilevel Queue (MLQ).
  - Multilevel Feedback Queue (MLFQ).
- Schemaläggning i multiprocessorsystem:
  - Symmetrisk och asymmetrisk.

Föreläsningen i ett nötskal

- Den här föreläsningen beskriver kriterierna vid schemaläggning av processorn och ett antal olika algoritmer för ändamålet.

- Föreläsningen motsvarar kapitel 5 *CPU Scheduling* i Operating System Concepts.

CPU-schemaläggning

- CPU-schemaläggning är centralt för operativsystem med time-sharing, multitasking.
- Det är den kortsiktiga schemaläggarens (*short-term scheduler*) uppdrag.

- Bestämmer vilken process som ska exekvera på processorn.
- Om operativsystemet stödjer flera kärntrådar per process så är det istället kärntrådar – inte processer – som schemaläggs.

CPU-schemaläggning
Dispatcher

- Den kortsiktiga schemaläggaren bestämmer alltså vilken process som ska allokeras till processorn härnäst.

- Utöver det finns också en *dispatcher* som utför själva context switchen. Den utför skiftet till användarläget och startar om processen på den korrekta instruktionen.

- Det är viktigt att dessa funktioner exekverar snabbt då de kanske körs med knappt millisekunder mellan varje switch.

CPU-I/O burst cycle

Processexekvering kan delas in i två klasser:

- CPU-burst: Processen exekverar på processorn.
- I/O-burst: Processen spenderar sin tid på att läsa från och skriva till olika enheter. Med andra ord väntar processen på I/O-enheter.

En process kör bara en av dessa två sprintar (bursts) om och om igen vilket skapar en cykel.

CPU-I/O burst cycle

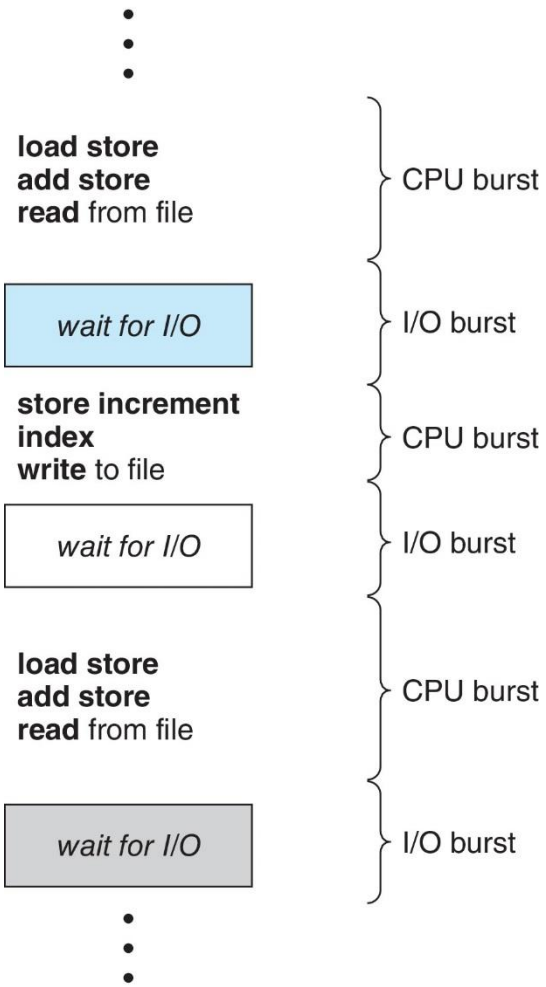

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 5.01

Typical CPU burst length

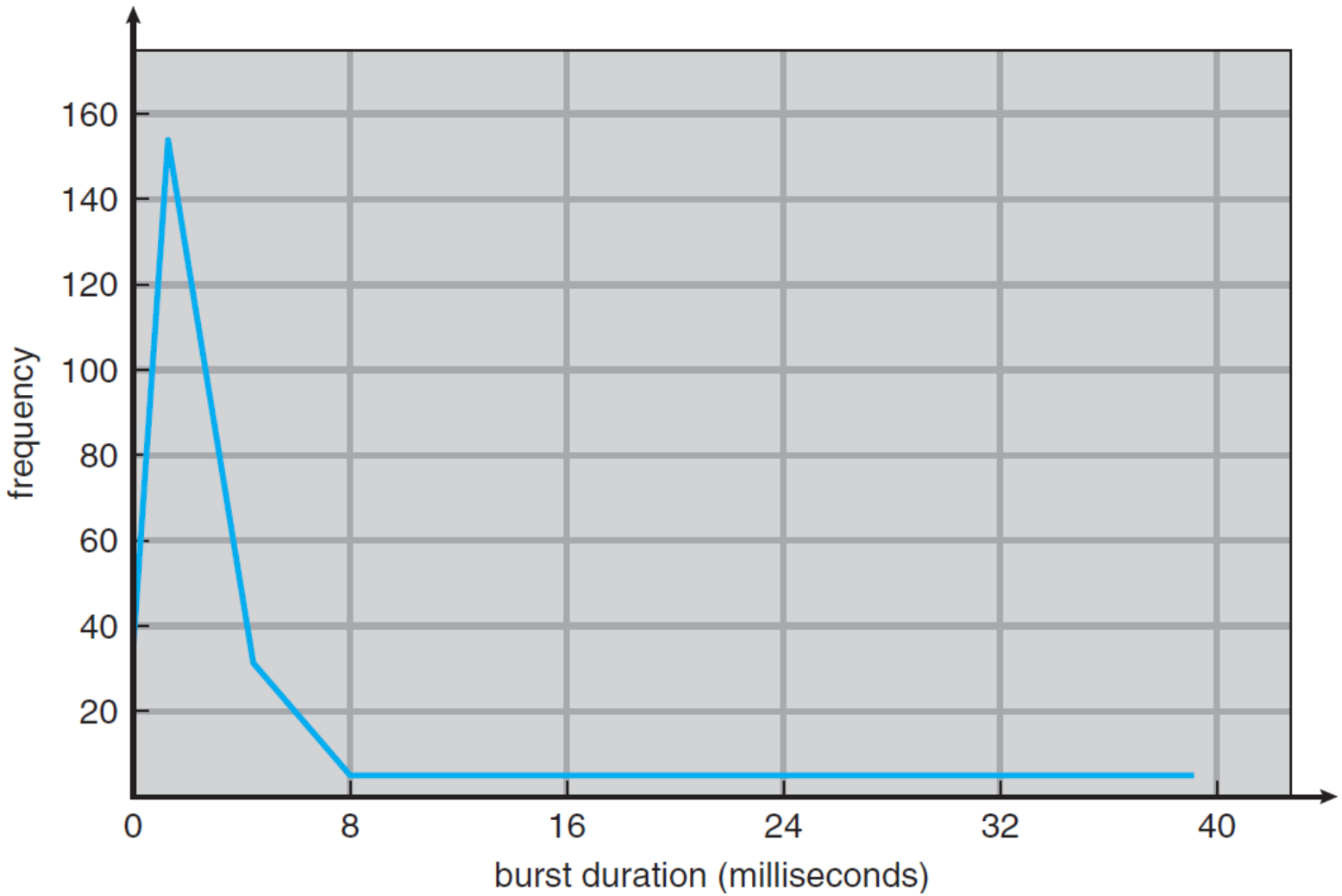

Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 6.02

Schemaläggning av processorn

Schemaläggning kan delas in i två klasser:

- Spärrande (*preemptive*).

- Icke-spärrande (*non-preemptive*).

Schemaläggning av processorn

Schemaläggning kan delas in i två klasser:

- Spärrande (*preemptive*).
  - Avbryter en process efter en given tidpunkt. T.ex. kan en process få 10 ms att exekvera under.
- Icke-spärrande (*non-preemptive*).
  - Kooperativ schemaläggning där processerna själva ger upp tillgången till processorn, t.ex. när de måste vänta på I/O.

Spärrande schemaläggning

- Spärrande schemaläggning är vanligast.
  - Krävs för interaktiva system.
- Den kräver emellertid speciell hårdvara, så som timers.
- Löper risk för synkroniseringsproblem som racetillstånd.

Kriterier vid schemaläggning

- CPU utilization — Vi vill hålla processorn så upptagen som möjligt.
- Throughput — Hur många processer blir klara per tidsenhet.
- Turnaround time — Tiden från att ett jobb antas till att det är färdigt.
- Response time — Hur lång tid det tar innan processen svarar.
- Waiting time — Tiden som spenderas väntandes i redokön. Ofta mätt som medelvärde.

Bild: A. Silberschatz, et al. Operating System Concepts, 10th global ed. 2019. s. 216-217.

First-Come, First-Served (FCFS)

- En enkel kooperativ algoritm som allokerar CPU:n till processerna enligt FIFO-principen.

+ Enkel att implementera.
- Generellt blir medelväntetiden hög. Konvojeffekten.

First-Come, First-Served (FCFS)

Process	Burst time (ms)
P_0	24
P_1	3
P_2	7

Tabell: Ett exempel på ett antal processer och deras sprintar (*bursts*).

- Vad blir väntetiden v_p för varje process?
- Hur lång blir medelväntetiden $\bar{v}$?
- Hur lång blir exekveringstiden x_{tot}?

Shortest Job First (SJF)

- Använder längden på processernas bursts för att bestämma exekveringsordningen. Kortast går först.
- Finns både spärrande och kooperativ varianter.

+ Ger en optimal medelväntetid. Med andra ord, så låg som möjligt.
- Det är nästintill omöjligt att uppskatta hur lång en burst kommer att vara. Längden på tidigare bursts kan användas för att ge en fingervisning.
- Risk för svält.

Shortest Job First (SJF)

Process	Burst time (ms)
P_0	24
P_1	3
P_2	7

Tabell: Ett exempel på ett antal processer och deras sprintar (*bursts*).

Prioritetsschemaläggning

- SJF är en form av prioritetsschemaläggning där prioriteten är inversen till den uppskattade längden av bursten.

- Prioritetsschemaläggning kan också generaliseras:
  - Högre prioriterade processer går före lägre prioriterade processer.
  - Processer med samma prioritet kan schemaläggas med FCFS.

- Det stora problemet med alla prioritetsbaserade algoritmer är svält.
  - Kan emellertid lösas med någon form av åldrande (aging).

Prioritetsschemaläggning

Process	Burst time (ms)	Prioritet
P_0	10	3
P_1	1	1
P_2	2	4

Tabell: Ett exempel på ett antal processer och deras sprintar (*bursts*) samt prioritet. Prioriteten kan sättas på olika sätt. Observera att höga värden inte alltid innebär hög prioritet.

Round Robin (RR)

- Använder tidskvantum. Varje process allokeras ett tidskvantum att exekvera under varpå den blir avbruten.
- Med ett tidskvantum q och n processer får varje process exekvera q tidsenheter och vänta som mest $(n - 1) \times q$ tidsenheter.
- Prestandan är beroende på storleken av tidskvantumet.

+ Spärrande schemaläggning krävs för interaktiva system.
- Ger generellt en hög medelväntetid.

Round Robin (RR)

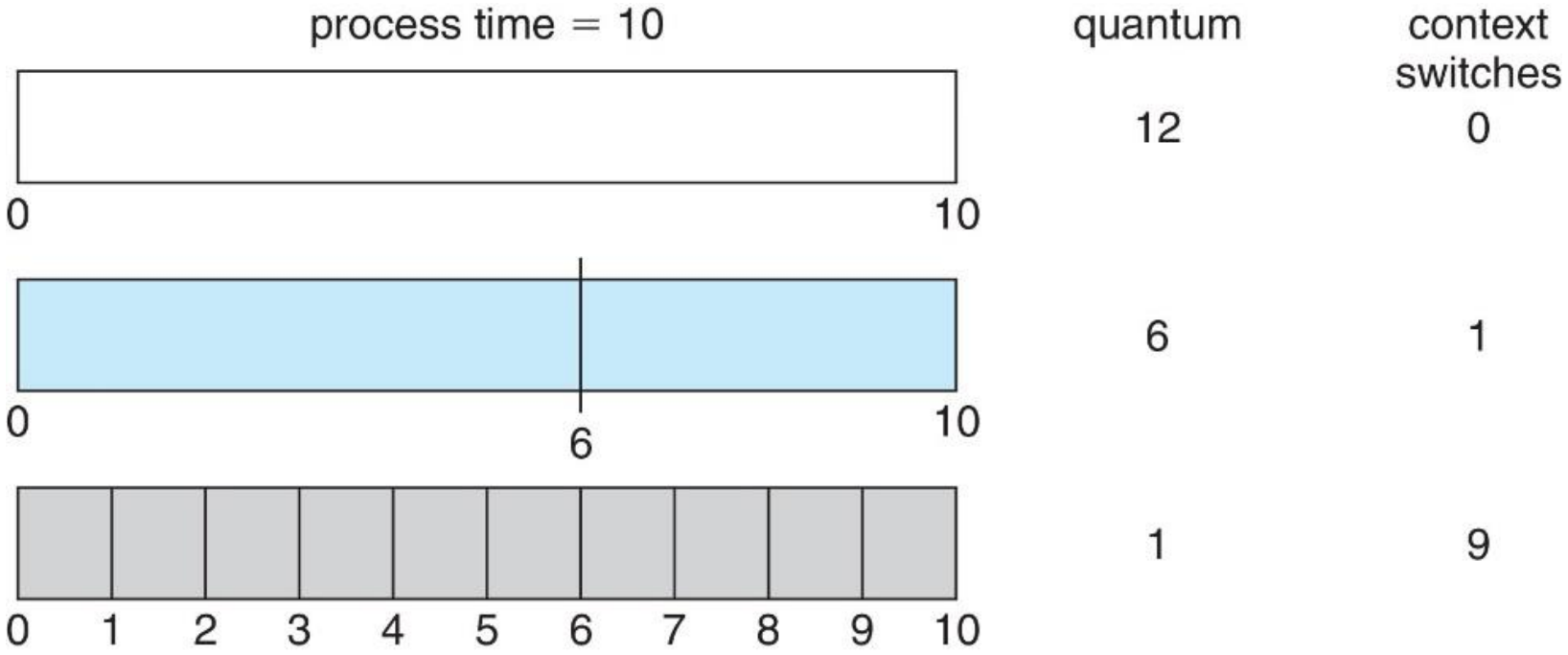

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 5.05

Round Robin (RR)

Process	Burst time (ms)
P_0	24
P_1	2
P_2	4

Tabell: Ett exempel på ett antal processer och deras sprintar (*bursts*). Anta tidskvantum q = 10 ms.

Multilevel Queue (MLQ)

- Vi kan ha flera redoköer.
- Varje kö kan köra sin egen schemaläggningsalgoritm.
- Utöver det kan vi välja hur vi schemalägger köerna.

- Vi kallar detta för en Multilevel Queue (MLQ).

- Processer i lågprioriterade köer avbryts när en process kommer in i en högre prioriterad kö.

Multilevel Queue (MLQ)

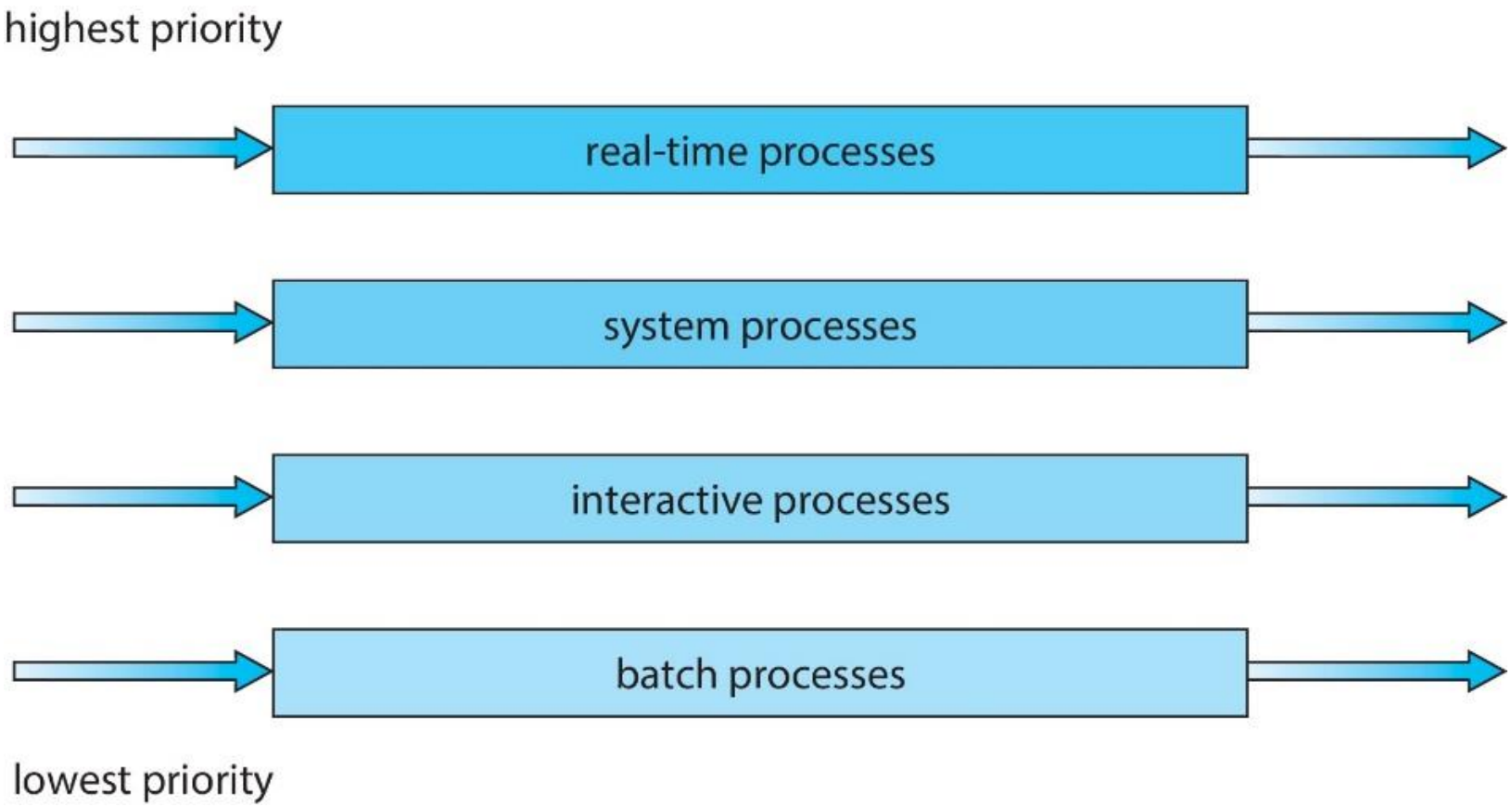

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 5.08

Multilevel Feedback Queue (MLFQ)

- Ett tillägg till MLQ där det nu går att flytta processer mellan olika köer baserat på deras CPU-burstmönster.
  - T.ex. kan en process som använder mycket CPU-tid sänkas i prioritet.
  - En process som arbetar mycket med I/O (och därmed relativt lite CPU) kan istället ökas.
  - När en I/O-intensiv process blir CPU-intensiv kan den sänkas igen.

Multilevel Feedback Queue (MLFQ)

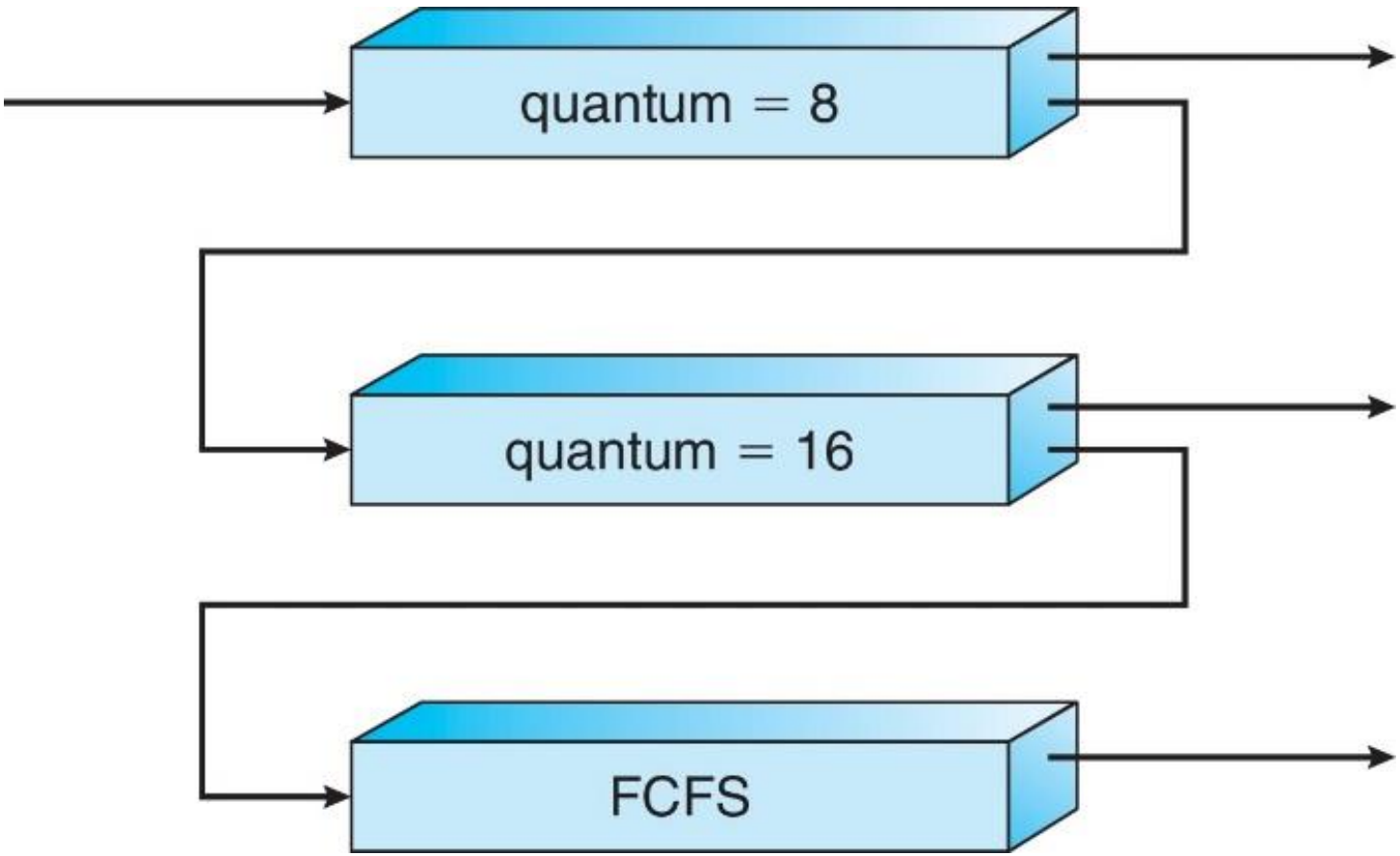

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 5.09

Schemaläggning i multiprocessorsystem Asymmetrisk

- Hur ska ansvaret fördelas med flera processorer?

- I den asymmetriska modellen har en processor huvudansvaret och delegerar arbetet till de andra processorerna.
- En processor utför då I/O, schemaläggning, med mera i kärnläge, medan alla andra exekverar i användarläge.

Schemaläggning i multiprocessorsystem Symmetrisk

- Symmetric multiprocessing (SMP) är den vanligare approachen.
- Alla processorer är likställda och delar på att exekvera allt, oavsett användar- eller kärnläge.

- Varje processor schemalägger sin egen exekvering. I vissa fall har de egna redoköer, i andra delade redoköer för samtliga processorer.

Symmetric multiprocessing (SMP)

- Ett problem med SMP är *non-uniform memory access* (NUMA). Det innebär att olika delar av primärminnet tar olika lång tid att komma åt för processorerna.

- Ett annat problem är *cache invalidation*. Om en process exekverar på en processor och sedan schemaläggs på en annan så är cachen ogiltig på den tidigare processorn och måste uppdateras.

- Det är alltså dåligt att flytta processer mellan processorer. Därav tillämpas ofta mjuk processortillhörighet (*soft processor affinity*).

Non-uniform memory access (NUMA)

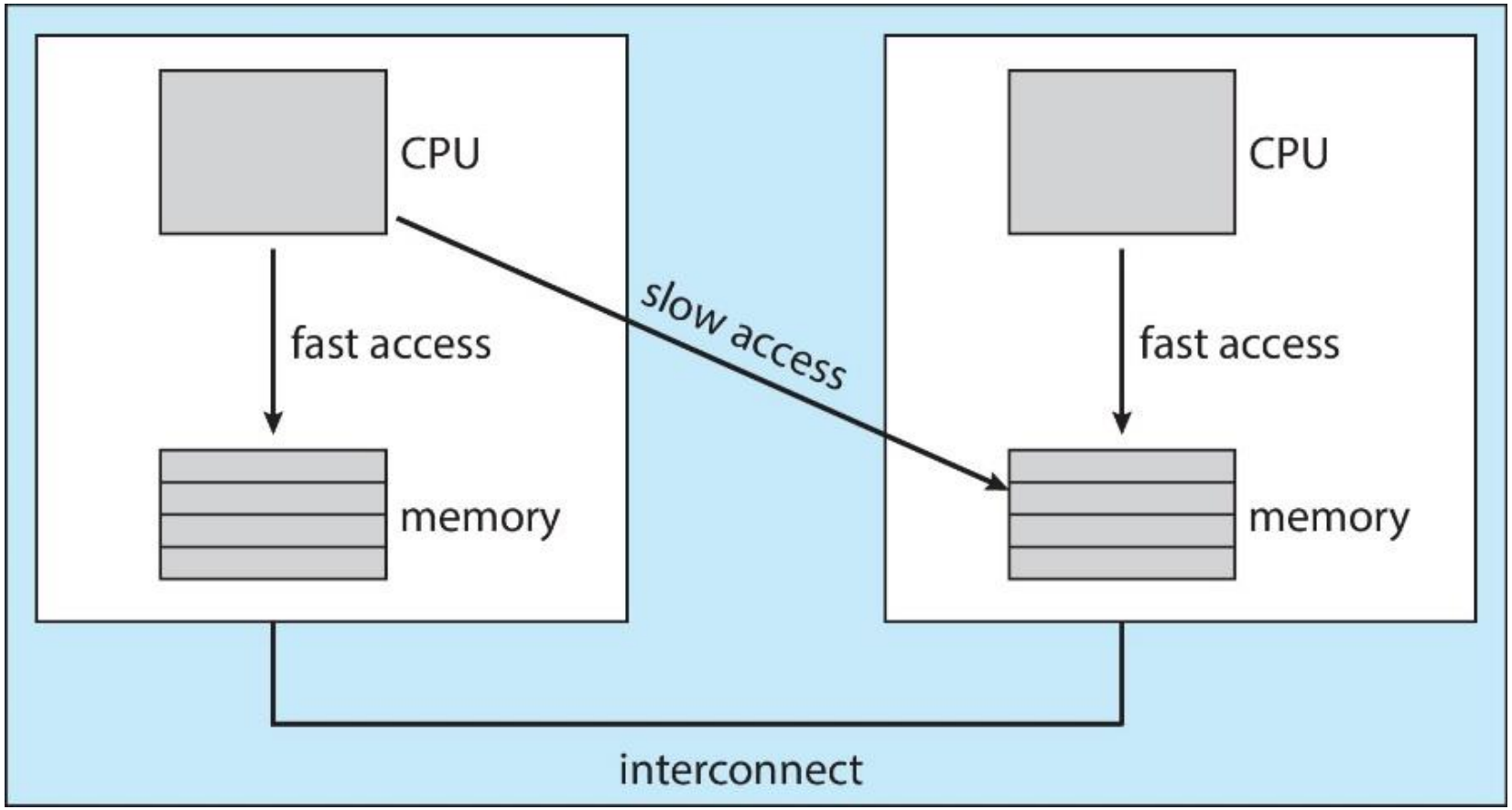

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 5.16

Symmetric multiprocessing (SMP)
Load balancing vs. processor affinity

- Processerna måste lastbalanseras mellan processorerna annars kan arbetsbördan bli skev.
  - Kan göras med push eller pull migration.
- Processerna har tillhörighet till processorer för att undvika att cachen invalideras.
  - Finns både mjuk och hård tillhörighet, som tillåter flyttar i vissa fall men inte andra.

- Att lastbalansera motverkar dock processortillhörigheten och leder till mer cacheinvalidering. Det är därför en viktig aspekt att avväga mellan de två när operativsystemet designas.

CPU-schemaläggning
Sammanfattning

- Processer exekverar i en CPU-I/O burst cycle och kan schemaläggas på processorn med spärrande eller kooperativ schemaläggning.
- Det finns olika kriterier vid schemaläggning där medelväntetiden är en av de viktigare.
- Olika algoritmer har olika ändamål och uppfyller kriterierna olika väl.